IoT-Based DIGITAL THERMOMETER

PROJECT REPORT

Submitted by

VISHAL V SHENOY (AM.SC.U4AID24051)

R. GIRINDRA (AM.SC.U4AID24069)

SURYA SURESH (AM.SC.U4AID24045)

SANAT SANTOSH NAIR (AM.SC.U4AID24038)

As a part of Project-based evaluation on 23ECE113
Introduction to Electronics

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI CAMPUS

JUNE 2025

TABLE OF CONTENTS

Chapter	Title	Page No.
1.	INTRODUCTION	01
2.	REVIEW OF LITERATURE	03
3.	DESIGN CONSIDERATION	05
4.	CIRCUIT DIAGRAM	11
5.	IMPLEMENTATION & RESULTS	12
6.	ADVANTAGES & APPLICATIONS	14
7.	CONCLUSION & FUTURE SCOPE	15
8.	REFERENCE	17

ABSTRACT

This project presents the design and implementation of an IoT-based digital thermometer system that integrates an LM35 analog temperature sensor with an Arduino Uno microcontroller and an ESP32 Wi-Fi module for efficient, real-time temperature monitoring and wireless data communication. The LM35 sensor continuously measures the ambient temperature and outputs an analog voltage proportional to the temperature, which is then fed into an operational amplifier for signal conditioning to enhance accuracy. The conditioned signal is read by the Arduino Uno, which processes and formats the data for serial transmission.

A voltage divider ensures proper logic-level compatibility as the Arduino transmits this data to the ESP32 module, which serves as the IoT interface by uploading the temperature readings to a designated cloud server. This enables remote access and monitoring of environmental conditions from any internet-connected device via a web-based dashboard.

An IRFZ44N MOSFET is employed to control power to the sensor, allowing the Arduino to manage energy consumption dynamically, thereby supporting low-power operation suitable for long-term deployments. The system adheres to standard IoT communication protocols, ensuring compatibility with broader IoT frameworks.

By combining the simplicity and accessibility of Arduino with robust signal processing, wireless communication, and energy-efficient design, this project demonstrates a scalable and reliable embedded solution for applications such as smart home systems, environmental monitoring, and industrial automation.

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

An IoT-based digital thermometer is a modern device that measures temperature and connects to the internet for easy access to data. Traditionally, thermometers are standalone devices that give you temperature readings on a display. However, with the IoT, these thermometers can send data to a smartphone, computer, or cloud service for real-time monitoring and analysis.

IoT-based thermometers are designed to offer more convenience and accuracy by allowing users to track temperature changes remotely. This technology is used in various applications such as healthcare, agriculture, and home automation. For example, in healthcare, doctors can monitor a patient's temperature from a distance, making it easier to track health status without needing physical checks.

These thermometers often use sensors to gather data and wireless communication like Wi-Fi to transmit information. The IoT-enabled system can provide alerts, store historical data, and help users make better decisions based on real-time insights. This innovation is transforming how we monitor temperature in many fields, making life more efficient and connected.

1.2 OBJECTIVES

The primary objective of an IoT-based digital thermometer is to provide real-time, accurate temperature monitoring and enable remote access to temperature data through internet connectivity.

- Remote Monitoring: Allow users to check temperature readings from anywhere using their smartphones or computers.
- Low Maintenance: IoT-based digital thermometers are designed to be low maintenance. Once set up, they can run autonomously with minimal need for manual handling, reducing effort and increasing reliability.
- High Cost Efficiency: Compared to traditional thermometers, IoT-based digital thermometers offer greater cost efficiency by reducing the need for manual monitoring and minimizing maintenance efforts. The integration of sensors with microcontrollers and wireless modules enables automated, continuous data collection and remote access, ultimately lowering operational costs and improving overall resource utilization.

- Facilitate Smart Integration: Assess the feasibility of integrating smart technologies, such as IoT-enabled monitoring and AI-based analytics, to improve system performance and user experience.
- Support Disaster Preparedness: Investigate the role of efficient rain detection systems in disaster management, such as early alerts for flooding or heavy rain

1.3 RELEVANCE

- **Real-Time Monitoring**: Continuously tracks temperature and updates data instantly.
- **Remote Access**: Users can view temperature readings from any location using the internet.
- **Smart Integration**: Can be connected to other IoT devices for automated climate control.
- **High Accuracy**: Offers precise measurements, improving reliability in critical applications.
- **Energy Saving**: Optimizes energy usage by enabling efficient system responses.
- **Compliance Support**: Helps maintain temperature-related standards in industries like food, pharma, and healthcare.

1.4 REPORT ORGANIZATION

This report is structured into several key sections, beginning with an introduction that outlines the project's background, objectives, and the importance of integrating IoT with digital thermometer systems for real-time, remote temperature monitoring. The Literature Review section explores existing temperature monitoring solutions, the role of IoT in environmental sensing, and prior works utilizing sensors like LM35 and platforms such as Arduino and ESP32 for wireless data transmission.

The System Design and Architecture section describes the overall circuit configuration, emphasizing the integration of the LM35 temperature sensor, Arduino Uno, operational amplifier, IRFZ44N MOSFET, and ESP32 module. The methodology section covers the hardware implementation based on the provided circuit diagram, including the wiring of components, configuration of the operational amplifier for signal amplification, and setup of the serial communication interface between the Arduino and ESP32. The Software Implementation section discusses the embedded programming done on the Arduino to read temperature values, control the MOSFET, and transmit data to the ESP32, which in turn handles wireless connectivity and cloud integration.

The Results section presents the observed temperature data, verifies the system's accuracy and stability, and notes any limitations encountered during testing. In the discussion, the system's performance is analyzed with respect to real-time monitoring capability, power efficiency via MOSFET control, and applicability in various real-world scenarios. Finally, the conclusion highlights the main outcomes, assesses the effectiveness of the proposed solution, and suggests future enhancements to further improve the scalability and reliability of IoT-based temperature monitoring systems.

CHAPTER 2
REVIEW OF LITERATURE

The project "IoT Based Contactless Thermometer" focuses on building a low-cost, non-contact temperature scanner using Arduino, IR sensor (MLX90614), and ESP8266 to send data to the cloud (ThingSpeak).The literature reflects the device's benefits in terms of affordability, cloud integration, and accuracy. However, limitations in IR thermometer precision are acknowledged, with future scope suggesting potential upgrades using tympanic thermometers. The proposed system aims to provide an accessible alternative to expensive commercial devices and promote public health through scalable, real-time temperature monitoring. **[1]**

The project titled "Digital Thermometer Using Arduino and MCP9808" explores the design of a precise temperature monitoring system using the MCP9808 digital sensor and Arduino UNO. The sensor offers high accuracy (±0.25°C) across a wide range (-40°C to +125°C), making it suitable for various applications. The system operates in two modes: standalone (data shown on an LCD) and PC-controlled (data sent to a C# application via COM port). Features include adjustable reading speeds using DIP switches or a GUI trackbar, and real-time graph plotting temperature in both Celsius and Fahrenheit. This literature showcases a practical and educational approach to building accurate, customizable, and user-interactive temperature sensors using microcontroller-based platforms. **[2]**

This project aims to design a digital thermometer using an Arduino and the high-precision MCP9808 temperature sensor. The MCP9808 offers ±0.25 °C accuracy and uses an I²C digital interface, ensuring noise-free, reliable readings without the need for analog conversion. Its real-time monitoring capability makes it ideal for sensitive applications, with easy integration into IoT systems for future wireless and cloud-based data access. **[3]**

This project focuses on developing a low-cost, accurate, and user-friendly digital thermometer using a microcontroller to monitor body temperature in real time and trigger an alarm when limits are exceeded. Using sensors like LM35 or DS18B20 with microcontrollers such as ATmega16 or Arduino UNO, these systems display readings on LCDs and may include modules for remote alerts. Simulated using tools like TinkerCAD or Multisim and programmed via Arduino IDE, these thermometers offer improved accuracy, real-time tracking, and added safety compared to traditional devices. **[4]**

This project aims to develop a smart digital thermometer using an LM35 sensor, Arduino UNO, and a 16x2 LCD for real-time, accurate temperature measurement. The LM35's analog output is processed by the Arduino and displayed in Celsius on the LCD. Built with basic, cost-effective components, the system is easy to implement and was successfully tested to provide precise and responsive room temperature readings. **[5]**

CHAPTER 3

DESIGN CONSIDERATIONS

3.1 COMPONENTS REQUIRED & SPECIFICATION:

SL.NO	Components	Quantity
1	Arduino	1
2	Wi-Fi Module (ESP32)	1
3	Resistor 1k	2
4	Resistor 2k	1
5	Resistor 220	1
6	Resistor 10k	1
7	N-Mosfet	1
8	LM35 Sensor	1
9	Opamp (LM741)	1
10	Breadboard	1

3.2 CALCULATION

LM35

$V_{out}=10 * 10^{-3} * T$

- where T is the input temperature

Assuming T to be 32°C

$V_{out}=10 * 10^{-3} * 32$ V

= 0.32V

Op-Amp (Non Inverting)

R_f=10

R_{in}=1

Ie A= 1+ R_f/R_{in}

Ie A=1+10/1

Ie A=11 where :

- R_f=> Feedback Resistance
- R_{in} => Input Resistance
- A => Amplification factor

Arduino

Assuming the input to be taken inside A0

The input voltage after amplification will be

0.32*11 = 3.52

For accounting for both the amplification and temperature conversion, the input is thus divided by 11 and multiplied by 100

Ie 3.52/11*100=32

Voltage Divider Equation

V_RX = 5V × (2.2k / (1k + 2.2k)

≈ 3.44V

3.3 DESIGN CONSIDERATIONS

WORKING OF LM35

The LM35 is a temperature sensor that outputs an analog signal that is proportional to the instantaneous temperature. The output voltage can easily be interpreted to obtain a temperature reading in Celsius. The advantage of the LM35 over the thermistor is it does not require any external calibration. The coating also protects it from self heating. Low cost (approximately $0.95) and greater accuracy make it popular among circuit makers, and students. Many low-end products take advantage of low cost, greater accuracy and used lm35 in their products.

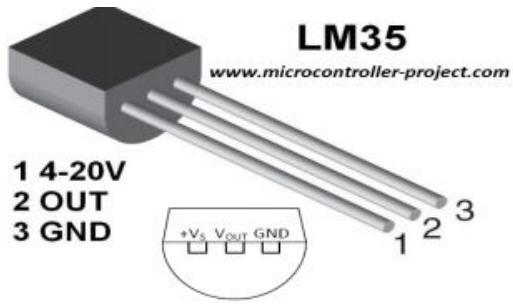

Fig 1. LM35 Pin Out diagram

LM35 can measure from -55 degrees centigrade to 150 degree centigrade. The accuracy level is very high if operated in at optimal temperature and humidity levels. Conversion of output voltage to centigrade is also easy and straightforward.

Input voltage to LM35 can be from +4 volts to 30 volts. It consumes about 60 microamperes of current. The LM35 has many family members; a few names are LM35C, LM35CA, LM35D, LM135, LM135A, LM235, LM335. All lm35 family members work on same principles, but temperature measuring capacity varies, and also they are available in many packages

LM35 Working Principle:

In order to understand the working principle of the LM35 temperature sensor, we have to understand the linear scale factor. The LM35 is designed to generate +10 mills volt per degree centigrade. It means that with increase in output of 10 mills volt by the sensor vout pin the temperature value increases by one. For example, if the sensor is outputting 100 mills at the Vout pin, the temperature in centigrade will be 10 degree centigrade. The same goes for the negative temperature reading. If the sensor is outputting -100 mills volt the temperature will be -10 degree Celsius.

IRF244N MOSFET

- Power Switching Efficiency: The IRF244N MOSFET acts as a switch to control the LM35's power, reducing consumption by enabling the sensor only during readings.

Gate Drive Requirement: A proper gate voltage (from the Arduino) ensures efficient switching, and a gate resistor (if needed) protects the MOSFET from overcurrent.

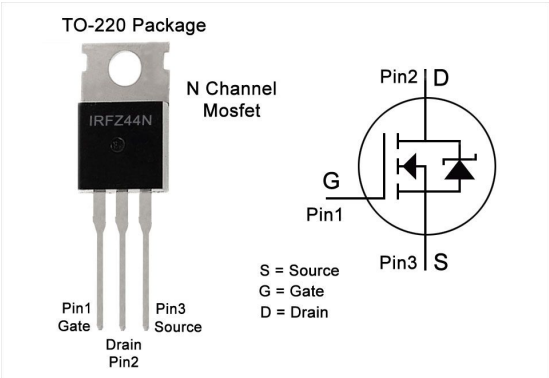

Fig 2. IRF244N Pin Out diagram

Operational Amplifier

- Signal Amplification: The op-amp amplifies the LM35's output voltage, improving resolution for the ESP32's ADC, with gain set by resistors R2 and R3.
- Output Voltage Scaling: The amplified output must be within the ESP32's 0–3.3V range to prevent damage, requiring careful selection of R2 and R3 values.

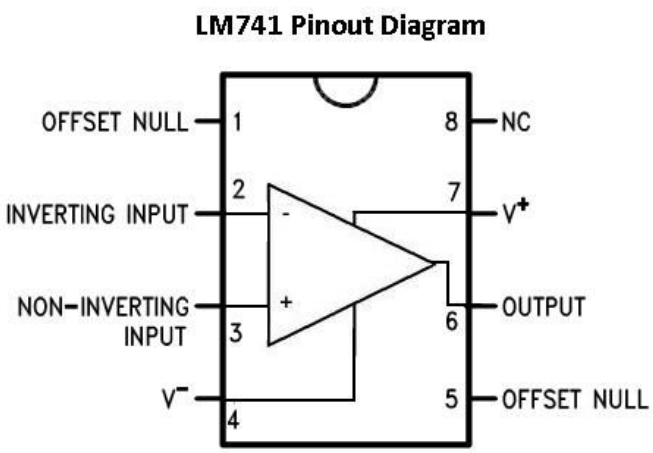

Fig 3. LM741 Pin Out diagram

3.4 ESP32 CODE

```
#define BLYNK_TEMPLATE_ID "TMPL37bfRQUIj"
#define BLYNK_TEMPLATE_NAME "ECEPROJECT"
#define BLYNK_AUTH_TOKEN "y9_Ky1s_74FWmCXSTYz2ag1JGCW6kWcT"
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
char ssid[] = "Arjuuu";
char pass[] = "12345678";
BlynkTimer timer;
String inputString = "";
float voltage = 0.0;
void readFromArduino() {
  while (Serial2.available()) {
    char c = Serial2.read();
    if (c == '\n') {
      String cleanStr = inputString;
      inputString = "";
      voltage = cleanStr.toFloat() * 1000/110
      Serial.println("Received: " + cleanStr); // for debugging
      Blynk.virtualWrite(V0, voltage);
    } else {
      inputString += c;
    }
}}
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, 16, 17); // RX=16, TX=17
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  timer.setInterval(100L, readFromArduino);
}
void loop() {
  Blynk.run();
  timer.run();
}
```

3.4 ARDUINO CODE

```
void setup() {
  Serial.begin(9600); // Serial to ESP32
}
void loop() {
  int raw = analogRead(A0); // 0–1023
  float voltage = raw * 5.0 / 1023.0;
  // Convert float to string with 2 decimal precision
  String data = String(voltage, 2);
  // Send full line once
  Serial.println(data);
  delay(1000);
}
```

3.5 CONCEPTS INVOLVED

Temperature-to-Voltage Conversion

● LM35 Output: The LM35 outputs an analog voltage proportional to temperature (10 mV/°C), which is read by the Arduino's A0 pin for further processing.

Analog-to-Digital Conversion (ADC)

● Arduino ADC: The Arduino Uno's 10-bit ADC converts the LM35's analog voltage (0–5 V) to a digital value (0–1023), with a resolution of 4.88 mV per step.

Serial Communication

● Arduino-ESP32 Link: The Arduino sends temperature data to the ESP32 via TX/RX pins, using resistors R4 and R5 to adjust voltage levels for safe communication.

IoT Data Transmission

● ESP32 Wi-Fi: The ESP32 processes the amplified temperature signal, converts it to digital, and transmits it wirelessly to a remote server for real-time monitoring.

CHAPTER 4

CIRCUIT DIAGRAM

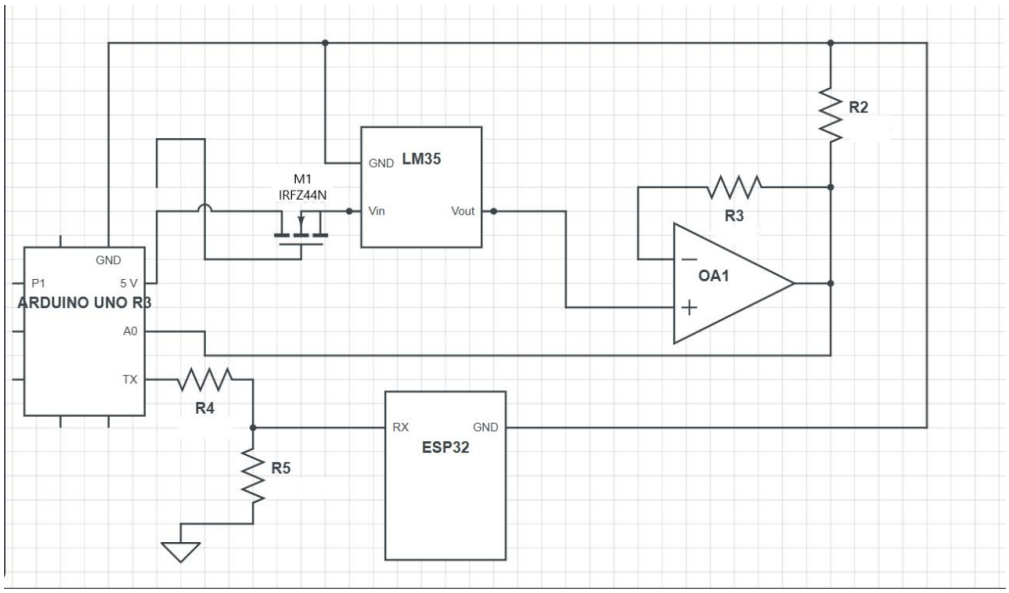

Fig 4. Schematic diagram of the circuit

This circuit is designed for an IoT-based digital thermometer, using an Arduino Uno R3 and ESP32 to measure, process, and transmit temperature data.

1. **Arduino Uno R3**: Acts as the main controller. It provides 5V power and ground (GND) to the circuit, reads the temperature data via its A0 pin, and sends data to the ESP32 through the TX pin for IoT connectivity.
2. **ESP32**: Receives data from the Arduino via its RX pin (serial communication). The ESP32 enables IoT functionality, likely transmitting temperature data wirelessly (via Wi-Fi or Bluetooth) to a remote server or app.
3. **LM35 Temperature Sensor**: Measures ambient temperature. Its Vin is powered by the Arduino's 5V, GND is grounded, and Vout generates an analog voltage (10mV/°C) proportional to the temperature, which is fed into the op-amp.
4. **Op-Amp (OA1)**: Configured as a non-inverting amplifier to boost the LM35's small output voltage for better accuracy. The gain is set by resistors R2 and R3 (gain = 1 + R2/R3). The amplified output is sent to the Arduino's A0 pin for analog-to-digital conversion.
5. **MOSFET (M1 - IRF244N)**: Likely used as a switch to control power to the LM35 or another part of the circuit, possibly for power-saving or reset functions.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 SOFTWARE IMPLEMENTATION

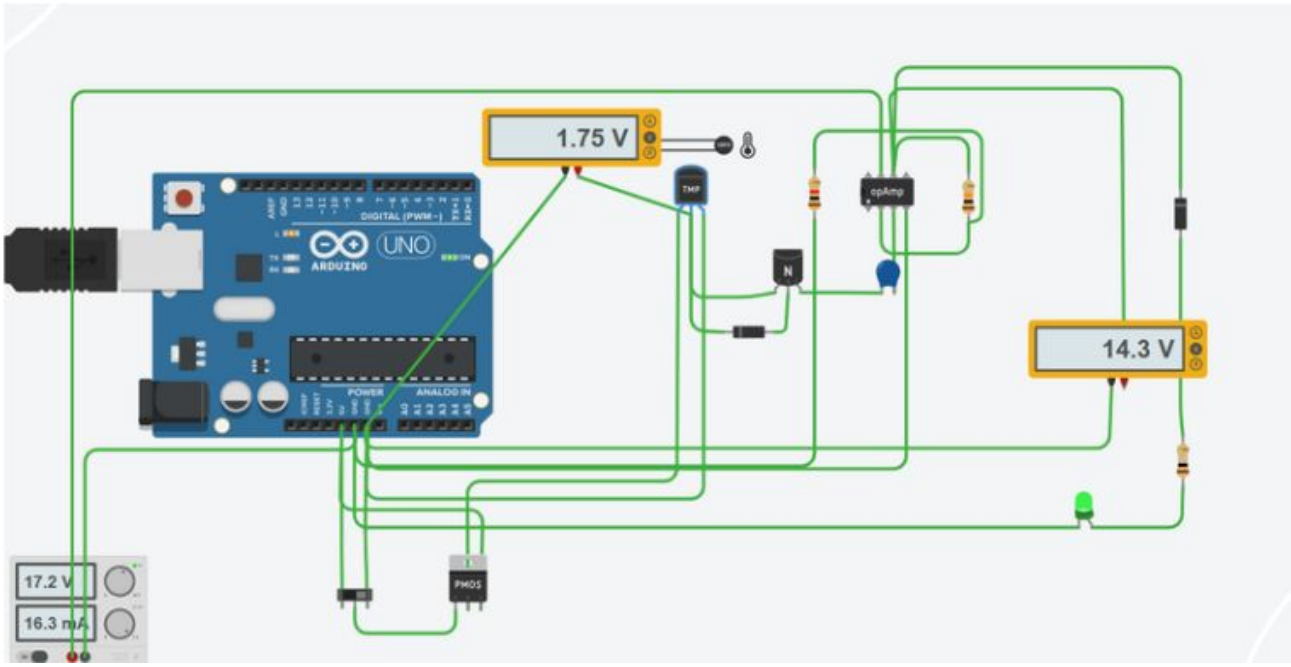

Fig 5. Software Implementation of circuit done in Tinkercad

This is a Tinkercad simulation of an IoT-based digital thermometer using an Arduino Uno, LM35 temperature sensor, and an ESP32 Wi-Fi module. The Arduino reads temperature data and activates an LED through an NMOS-NPN transistor setup when a threshold is crossed. An op-amp ensures accurate signal conditioning.

The ESP32 sends real-time temperature data to platforms like Blynk, enabling remote monitoring. This compact system is ideal for smart homes, environmental monitoring, and automated temperature control.

5.2 HARDWARE IMPLEMENTATION

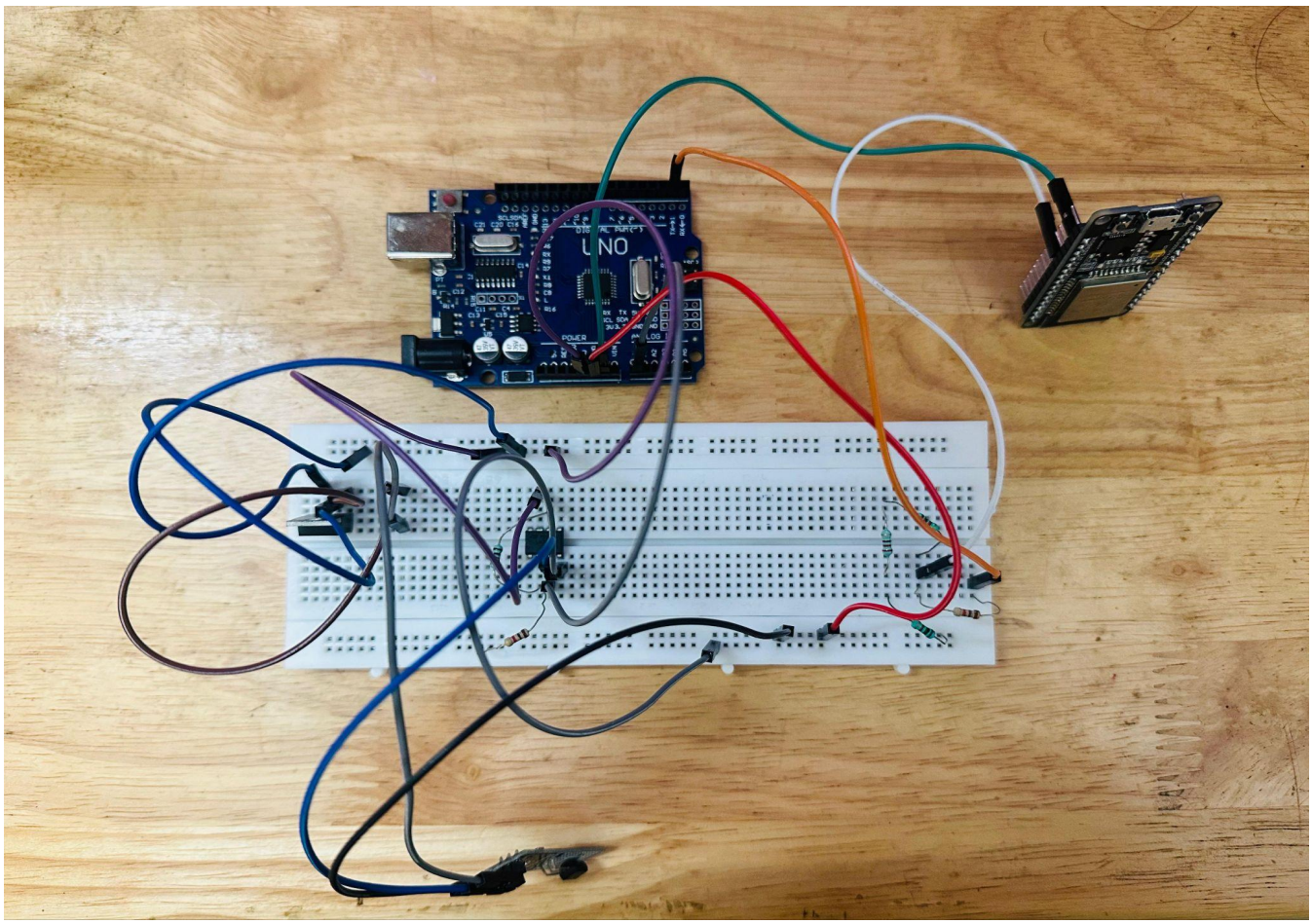

Fig 6. Hardware implementation of circuit

This image shows the hardware implementation of the IoT-based digital thermometer system on a breadboard. The setup includes an LM35 temperature sensor, an operational amplifier (Op-Amp), an IRFZ44N MOSFET, an Arduino Uno, and an ESP32 Wi-Fi module. The LM35 sensor continuously senses the ambient temperature and outputs a corresponding analog voltage, which is conditioned through the Op-Amp for accuracy. The Arduino Uno reads this voltage via its analog input pin and processes the data. The IRFZ44N MOSFET is used to control the power to the sensor, enabling efficient power management. The processed temperature data is then transmitted via serial communication to the ESP32, which sends it wirelessly to a cloud server. This implementation demonstrates real-time temperature monitoring and wireless data transmission, enabling remote access and visualization through a web dashboard.

CHAPTER 6

ADVANTAGES & APPLICATIONS

6.1 ADVANTAGES

1. **Remote Monitoring:** Enables real-time temperature monitoring from any location using IoT connectivity.
2. **Automation & Alerts:** Automatically records and sends alerts when temperature thresholds are exceeded.
3. **Energy Efficient:** Can be powered with low-energy components or solar for long-term use.
4. **Cost-Effective Components:** Uses commonly available and inexpensive components like resistors, MOSFETs, and LEDs.
5. **Immediate Visual Feedback:** LED indicator provides instant status indication without needing a screen.

6.2 APPLICATIONS

1. **Fan Controller—**It can turn on a fan automatically when the temperature goes too high.
2. **Overheat Alarm—**It can alert you when a device gets too hot.
3. **Greenhouse Monitor—**Helps keep plants safe by managing temperature inside greenhouses.
4. **Battery Safety System—**Keeps batteries from overheating by shutting down the system.
5. **Machine Protection—**Monitors machines and stops them if they get too hot.
6. **Smart Home Use—**Can be used in smart homes to control appliances based on temperature.

CHAPTER 7

CONCLUSION & FUTURE SCOPE

7.1 CONCLUSION:

The IoT-based digital thermometer project successfully demonstrates the effective integration of analog temperature sensing with IoT technology, forming a reliable system for continuous, real-time temperature monitoring and remote data access. Utilizing the LM35 temperature sensor, an operational amplifier for signal conditioning, the Arduino Uno for data processing, and the ESP32 module for wireless communication, the system ensures accurate and consistent temperature tracking. The analog signal from the LM35 is amplified to enhance precision, while the Arduino reads and formats the data for serial transmission. A voltage divider ensures compatibility between the Arduino and ESP32, enabling seamless Wi-Fi-based data transmission to cloud platforms for remote visualization via a web dashboard.

Challenges such as signal stability, voltage level mismatches, and power efficiency were addressed through careful circuit design—specifically, by incorporating an IRFZ44N MOSFET for controlled sensor power and by conditioning the sensor output for cleaner analog input. The modular hardware setup allows for future scalability, such as integrating additional sensors for humidity or air quality monitoring. Testing across varied conditions confirmed the system's robustness, low power consumption, and reliable data communication, validating its potential for real-world applications in home automation, environmental sensing, and industrial monitoring.

This project highlights how simple electronic components, when combined with microcontrollers and IoT modules, can be engineered into scalable and energy-efficient smart systems. The successful implementation of this digital thermometer sets a strong foundation for expanding into more complex multi-sensor networks that contribute to smarter, more responsive environments in healthcare, agriculture, and industry.

7.2 FUTURE SCOPE:

An IoT-based digital thermometer using an Arduino Uno, ESP32, and LM35 sensor measures temperature and sends data over the internet for real-time monitoring. Its future scope includes enhancing remote healthcare, smart homes, industrial safety, and public health by integrating with AI, wearables, and energy-efficient systems.

- Remote Health Monitoring:

  Connect to apps for real-time temperature tracking, alerting doctors for fever spikes in patients.
- Smart Environment Control:

  Integrate with home systems to adjust heating/cooling based on body or room temperature.
- Public Safety:

  Deploy in public spaces (e.g., schools, airports) to track temperatures and detect outbreaks.
- AI Predictions:

  Use AI to analyze temperature data and predict health issues like infections early.
- Energy Efficiency:

  Improve battery life with low-power IoT protocols and use sustainable materials.
- Industrial Safety:

  Monitor temperatures in factories, sending alerts to prevent equipment damage or product spoilage.

REFERENCES

[1] M.S. Irshath Ahmed, B. Joe Jasper Raja, R. Subhasan, and K.P. Saravanakumar, "Student Journal of Electrical and Electronics Engineering," vol. 1, no. 1, 2022.

[2] N. S. Patil, "International Journal of Research Publication and Reviews (IJRPR)," vol. 3, no. 3, pp. 1863-1866, Mar. 2022.

[3] P. Sonawane, T. Dudhal, S. Kendhale, S. Wankhede, and V. Sule, "International Research Journal of Modernization in Engineering Technology and Science," Int. Res. J. Mod. Eng. Tech. Sci., vol. 3, no. 8, Aug. 2021.

[4] S. T. Srikanth, D. B. Dhanalakshmi, A. D. Amuktha, M. J. Manikanta, R. T. Ramalokeswar, and N. P. Nagaphanindhra, "Portable Rain Water Detecting Alarm Using IC 555 Timer," South Asian Journal of Engineering and Technology, Vol. 12, No. 2, 2022.

[5] M. A. Hassan and H. M. Abdullahi, "Dept. of EEE, Gombe State Polytechnic Bajoga, Nigeria."